Security Analysis Report: Metasploit Automation Wrapper Scripts

Subject: Bash-based menu wrappers around the Metasploit Framework (payload generation, listener setup, persistence, Wi-Fi auditing) **Report type:** Defensive / detection-focused analysis **Audience:** Security team, SOC analysts, blue-team, incident responders

---

1. Purpose of this report

This report explains, at a conceptual and structural level, what a class of tools this script belongs to does and — more importantly — **how a defender detects, contains, and investigates them**. It intentionally does not walk through exact commands, flags, or payload syntax; the goal is detection and response readiness, not reproduction.

---

2. What Metasploit is, conceptually

Metasploit is a widely used, dual-use **penetration-testing framework**. “Dual-use” means the exact same components that a licensed tester uses in an authorized engagement are what an attacker uses in a real intrusion — the software doesn’t distinguish intent, only authorization and scope do.

At a high level, Metasploit is organized around a few categories of component:

Category	What it means conceptually
Payloads	Code that runs on a target machine after successful exploitation or execution, typically to open a communication channel back to the operator
Handlers / listeners	A process on the operator’s machine that waits for a payload to “call home” and then manages that connection
Exploits	Modules that take advantage of a specific vulnerability to get a payload to run on a target in the first place
Post-exploitation modules	Actions taken *after* access is gained — harvesting data, escalating privilege, or maintaining access
Persistence mechanisms	Techniques to make access to a compromised system survive a reboot, logout, or process kill

A “wrapper script” like the one under review doesn’t add new capability to Metasploit — it automates the menu of choices an operator would otherwise type by hand, and organizes the output into a folder for reuse. This matters for detection: **the risk indicators are the same as manual Metasploit use**, just produced faster and more consistently.

---

3. What this class of script automates (categorically)

Based on the structure of the uploaded script, it is a menu-driven wrapper that automates four

categories of activity, plus an unrelated Wi-Fi auditing tool. Described generically:

3.1 Payload generation

Generates platform-specific reverse-connection payloads (Windows, Linux, macOS, Android) using Metasploit's payload generator, and saves the output files to a fixed local folder. A "reverse" payload means the *target* machine initiates the outbound connection to the operator, which is significant for detection (see §5).

3.2 Multi-handler / listener setup

Writes a Metasploit "resource file" (a short script of console commands) that configures a listener matching the payload type, then launches the Metasploit console non-interactively with that file. This is the piece that "catches" a payload once it runs on a target and calls back.

3.3 Persistence

Includes a module intended to make access to a compromised device durable — in the sample reviewed, this takes the form of a loop script for Android that periodically relaunches a malicious component. Persistence is one of the highest-signal categories for defenders because **legitimate software rarely needs to re-launch itself via a covert loop**.

3.4 Wi-Fi auditing (Wifite)

Invokes a separate, unrelated tool for automated wireless network attacks (handshake capture / cracking workflows). This is bundled into the same menu but is architecturally unrelated to the Metasploit components above.

3.5 Exploitation via the framework's console

A placeholder in the reviewed script for launching Metasploit's console directly to run exploit modules against a target — the step that would actually deliver a payload to a vulnerable system.

---

4. Why this pattern is significant, defensively

A single-purpose exploit script is a known, bounded risk. A **menu-driven wrapper that chains payload generation → listener → exploitation → persistence** is more significant because:

- It lowers the skill floor: a user doesn't need to understand each Metasploit command, just pick a number from a menu.
- It standardizes artifact names and locations (e.g., a fixed output directory), which is good for defenders *if* they know the pattern, but makes mass replication trivial for an attacker.
- Bundling persistence with payload generation signals intent to maintain long-term access, not just demonstrate a one-time proof of concept.
- Requiring root to run, and using `xterm` to spawn interactive consoles, are both consistent with intent to operate outside normal monitored application channels.

---

5. Detection guidance for defenders

5.1 Host-based indicators

Indicator	Why it matters
New executables with generic names (`autorun.exe`, `autorun.elf`, `autorun.apk`, `autorun.macho`) appearing in user-writable directories	Common default output names from automated payload generators
A directory unexpectedly created and populated under a user's Desktop or home folder, containing several platform-specific binaries at once	Consistent with batch payload generation rather than normal software installation
Framework console or payload-generator processes spawned by a script rather than an interactive terminal	Automation wrappers invoke these components programmatically
`.rc` (resource) files referencing `exploit/multi/handler`, `set PAYLOAD`, `set LHOST`, `set LPORT`	These are Metasploit console command scripts — their presence indicates listener configuration, even without running them
A shell script that loops indefinitely, periodically invoking `am start` (Android) or scheduling itself via `cron`, `systemd` timers, `launchd`, or Windows Scheduled Tasks	Classic persistence-loop pattern
Requirement to run as root/administrator for what claims to be a "utility" script	A red flag when the stated purpose doesn't need elevated privileges

5.2 Network-based indicators

- **Outbound connections initiated by internal/endpoint hosts to external IPs on unusual or attacker-chosen ports**, especially shortly after a new or unexplained binary appears on the host — this is the signature of a reverse-shell payload calling back to a handler.
- Long-lived, low-traffic TCP sessions that don't map to any known application — meterpreter-style channels often sit idle waiting for operator input.
- Repeated reconnection attempts at fixed intervals to the same external host/port — consistent with a persistence loop trying to re-establish a session after it drops.
- On Android specifically, unexpected outbound traffic correlated with an app component being relaunched via intent broadcasts on a fixed timer.

5.3 Process and behavioral indicators

- A terminal emulator (e.g., `xterm`) spawned by a script rather than by a user, immediately running a security tool with `-r <resource-file>` arguments.
- Wireless interface being placed into monitor mode outside of scheduled, authorized wireless assessments.
- Privilege check logic at the very start of a script (`id -u -eq 0`) — common in tooling designed to operate with full system access from the outset.

5.4 Recommended controls

1. **Application allow-listing / EDR alerting** on the framework's console, payload-generator, and wireless-auditing tool execution outside of an approved, documented pentest engagement window.
2. **Egress filtering** — block or alert on outbound connections to non-business destinations on arbitrary high ports from end-user endpoints.
3. **File-integrity / creation monitoring** on user directories for newly created executables matching common payload-generator output patterns.
4. **Persistence-mechanism auditing** — regularly review scheduled tasks, cron jobs, systemd timers, launch agents, and (for MDM-managed Android fleets) unexpected auto-launch configurations.
5. **Root/administrator execution logging** — alert when scripts of unknown provenance request or obtain elevated privileges.

6. **Formal rules of engagement** for any legitimate internal red-team or pentest use, with a change ticket or engagement ID that can be correlated against the indicators above to distinguish authorized testing from unauthorized use.

---

6. What legitimate, authorized use looks like

Tools in this category (Metasploit and its automation wrappers) have a real place in **authorized** security work:

- Internal red teams validating detection coverage under signed rules of engagement
- Licensed penetration testers operating within a defined, client-approved scope
- Security researchers in isolated lab environments with no route to production systems

In every legitimate case, the defining features are: **written authorization, a bounded and documented scope, an isolated or explicitly in-scope target environment, and coordination with the defensive team** (or the defensive team *is* the one running it, purple-team style). None of those are properties of the script itself — they're organizational controls that have to exist around it.

---

7. Summary

The reviewed script is a convenience wrapper that automates existing Metasploit and Wifite capabilities behind a menu: payload generation across four platforms, listener setup, a persistence mechanism, and Wi-Fi attack automation, gated behind a root check. Its risk profile is identical to manual use of the underlying tools, and defenders should focus detection effort on the categorical indicators in §5 — new payload-shaped binaries, outbound reverse-connection traffic, Metasploit resource-file artifacts, and persistence-loop behavior — rather than on this specific script, since the same indicators apply to any tool in this category.